



数据及预处理

2026/4/2

数据是什么？

- 样本和属性的集合
- 属性（Attribute）是一个样本的特性或特征
 - 例如: 性别、年龄、气温
 - Attribute is also known as variable（变量）, field（域）, characteristic（特征）, or feature（特征）
- 一组属性用来描述一个样本（Object）
 - Object is also known as record（记录）, point（点）, case（事例）, sample（抽样）, entity（实体）, or instance（实例）

属性

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

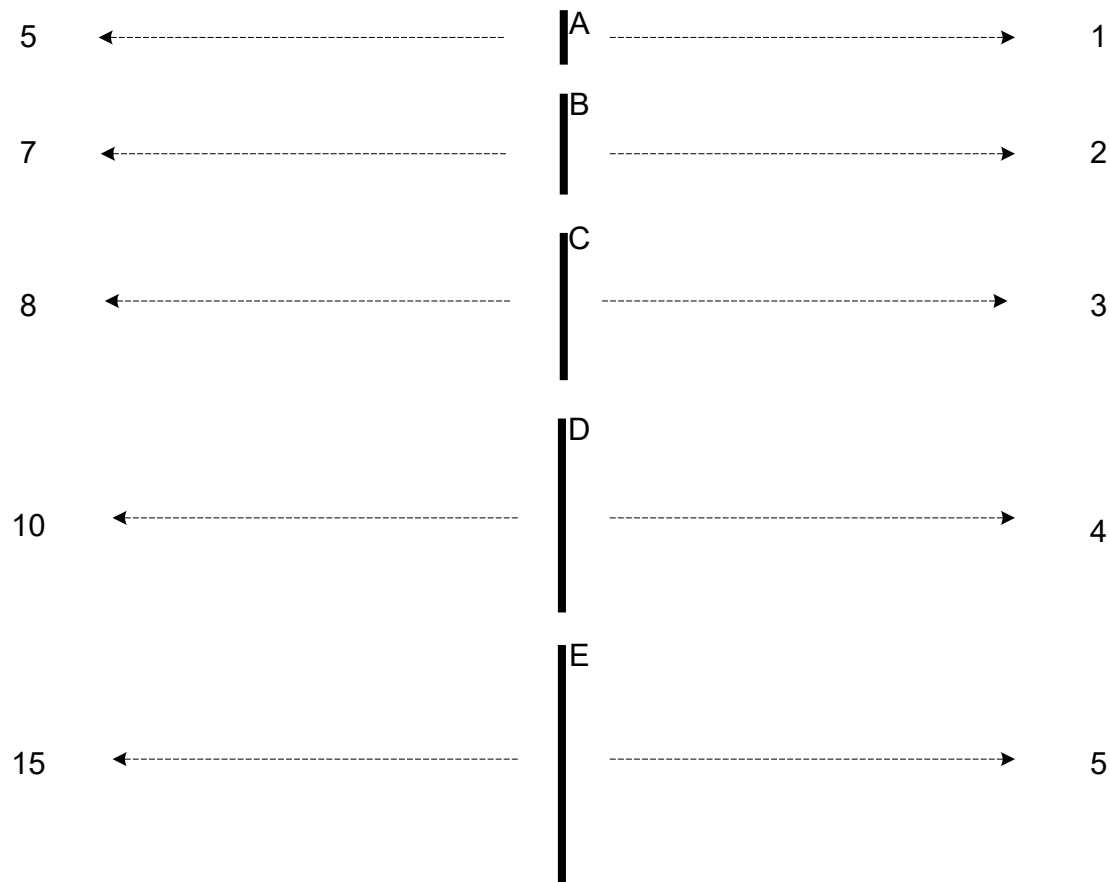
样本

属性值

- 属性值是赋给一个属性的“数值（number）”或“符号（number）”
- 属性和属性值的区别
 - 同一个属性可以用不同的属性值来表示
 - 如：长度可以以米、里或公里为单位来表示
 - 不同的属性可以用相同的属性值来表示
 - 如：ID和年龄的属性值都是实数
 - 但属性的含义可能不一样
 - ID取值没有限制，但年龄取值得在一定范围
 - 对于ID的属性来说，“12”可能表示第12格；但对于年龄的属性，“12”则意味着12岁

同一个属性可以用不同的方式测量

■ 属性可以用一种不描述属性全部性质的方式测量



两种不同的测量标度下的线段长度测量值

离散属性和连续属性

■ 离散属性

- 只包含有限个或无限可数个值
- 如：邮政编码，计数，文档中的词
- 一般表达为实数变量
- 注：二值属性是离散属性的特例

■ 连续属性

- 实数作为属性值
- 如：温度，高度，重量
- 实际上，实值只能使用有限数字来测量和表示
- 连续属性通常表示为浮点变量

数据集类型

■ 记录

- 数据矩阵
- 文档数据
- 事务数据

■ 基于图形的数据

- 万维网
- 分子结构

■ 有序数据

- 空间数据
- 时间数据
- 时序数据
- 基因组序列数据

数据矩阵

- 如果一个数据集族中的所有数据对象都具有相同的数值属性集，则数据对象可以看做多维空间中的点，其中每个维代表对象的一个不同属性。
- 这样的数据对象集可以用一个 $m \times n$ 的矩阵表示，其中 m 行，一个对象一行； n 列，一个属性一列。

Projection of x Load	Projection of y load	Distance	Load	Thickness
10.23	5.27	15.22	2.7	1.2
12.65	6.25	16.22	2.2	1.1

文档数据

■ 文档可用词向量表示

- 每个词是向量的一个分量
- 每个分量的值是对应词在文档中出现的次数

	team	coach	play	ball	score	game	win	lost	timeout	season
Document 1	3	0	5	0	2	6	0	2	0	2
Document 2	0	7	0	2	1	0	0	3	0	0
Document 3	0	1	0	0	1	2	2	0	3	0

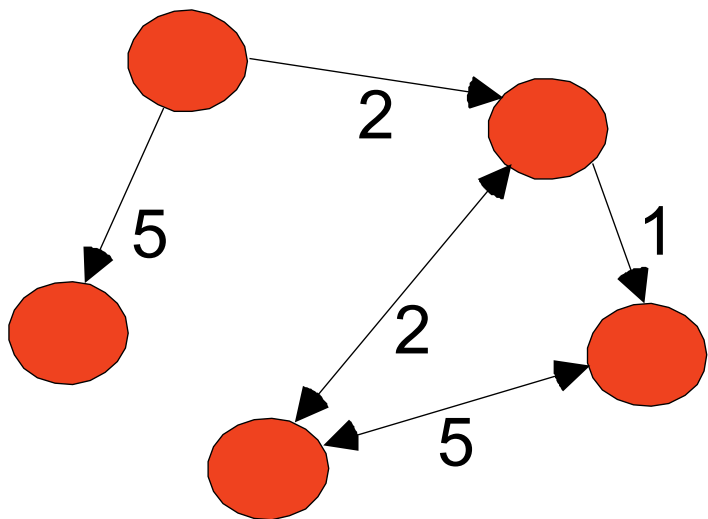
事务数据

- 一种特殊类型的记录数据, 其中
 - 每个记录涉及一系列的项
 - 考虑一个杂货店。顾客一次购物所购买的商品的集合就构成一个事务，而购买的商品是项。

<i>TID</i>	<i>Items</i>
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

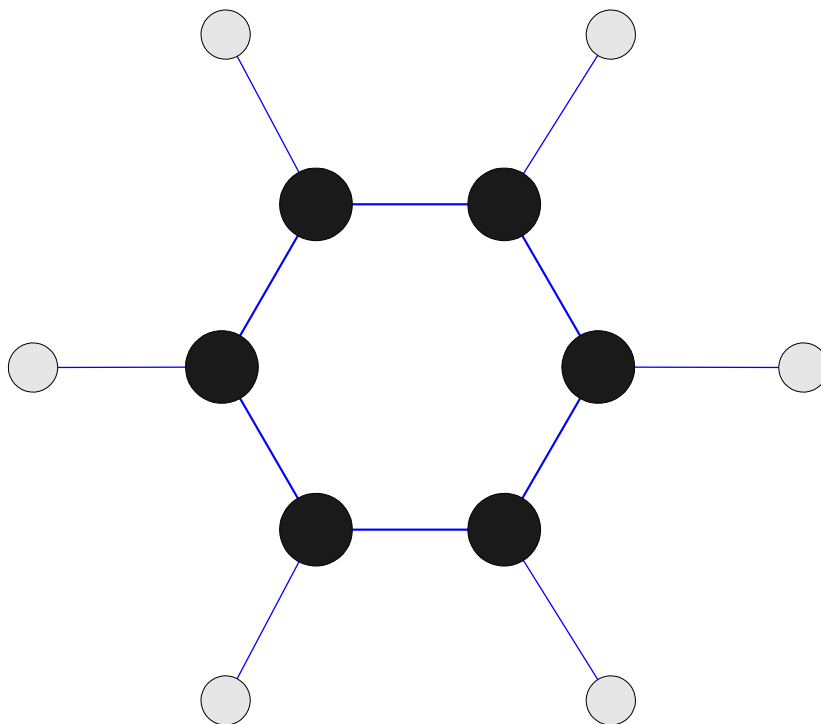
基于图形的数据

- 如: 通用图和HTML链接



```
<a href="papers/papers.html#bbbb">  
Data Mining </a>  
<li>  
<a href="papers/papers.html#aaaa">  
Graph Partitioning </a>  
<li>  
<a href="papers/papers.html#aaaa">  
Parallel Solution of Sparse Linear System of Equations </a>  
<li>  
<a href="papers/papers.html#ffff">  
N-Body Computation and Dense Linear System Solvers
```

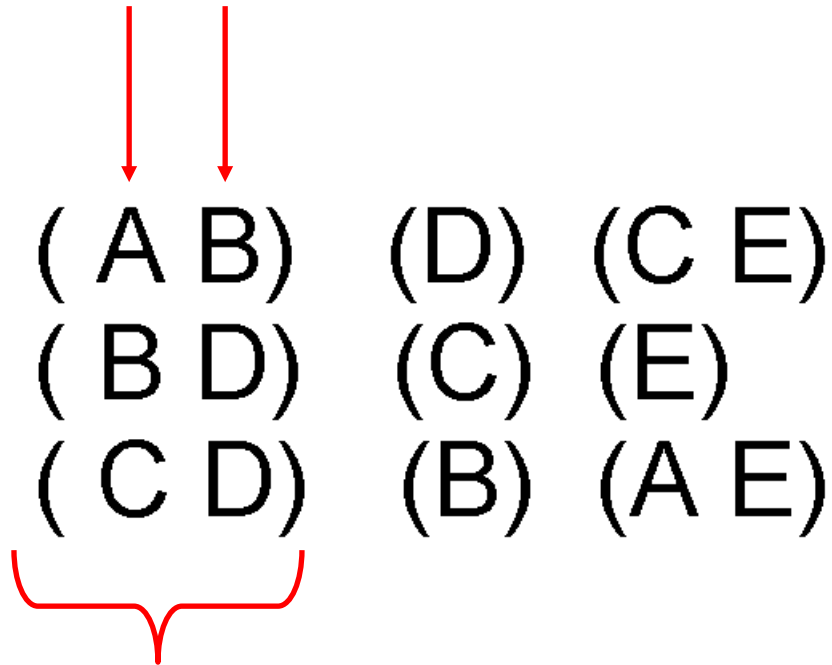
■ 苯分子: C_6H_6



有序数据

■ 时序事务数据

Items/Events



An element of
the sequence

有序数据

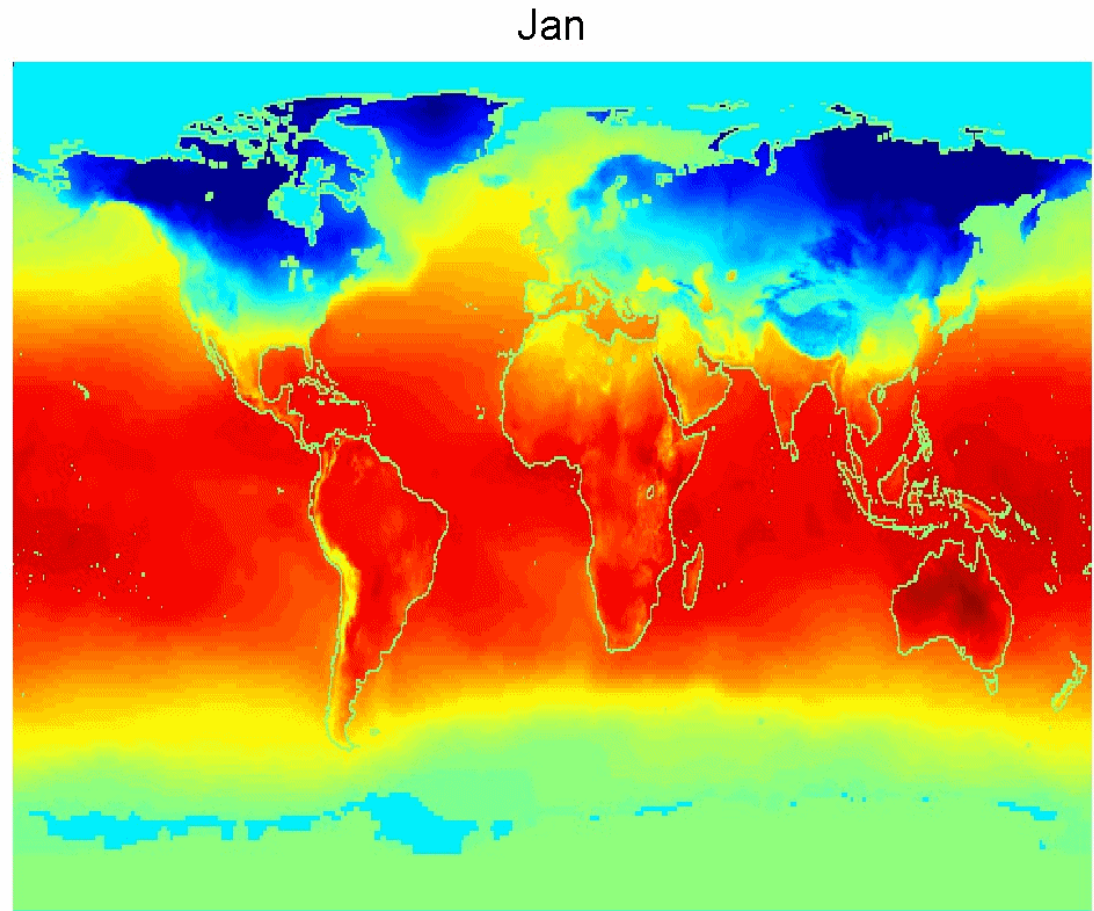
■ 基因组序列数据

**GGTTCCGCCTTCAGCCCCGCGCC
CGCAGGGCCCGCCCCGCGCCGTC
GAGAAGGGCCCGCCTGGCGGGCG
GGGGGAGGCGGGGCCGCCCGAGC
CCAACCGAGTCCGACCAGGTGCC
CCCTCTGCTCGGCCTAGACCTGA
GCTCATTAGGCGGCAGCGGACAG
GCCAAGTAGAACACGCGAAGCGC
TGGGCTGCCTGCTGCGACCAGGG**

有序数据

■ 空间温度数据

陆地和海洋的平均
月温度



相似性和相异性

■ 相似度

- 两个对象相似程度的数值度量.
- 两个对象越相似，相似度越高
- 在区间 $[0,1]$ 取值

■ 相异度

- 两个对象差异程度的数值度量
- 相异度的最小值通常是0
- 上限不同

■ 接近度是指相似度或相异度

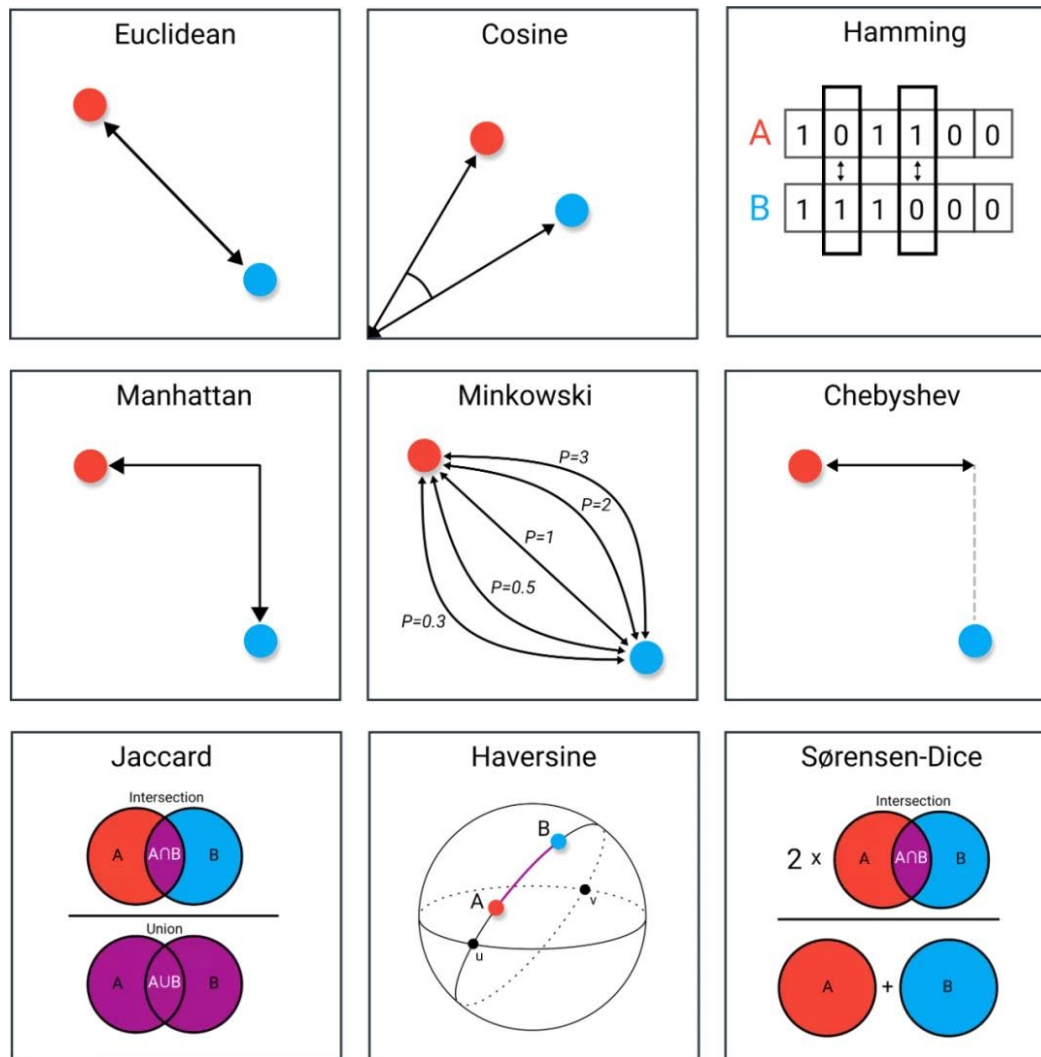
- 距离，例如欧几里得距离，有一些常见属性：

1. $d(p, q) \geq 0$ for all p and q and $d(p, q) = 0$ only if $p = q$. (Positive definiteness)
2. $d(p, q) = d(q, p)$ for all p and q . (Symmetry)
3. $d(p, r) \leq d(p, q) + d(q, r)$ for all points p, q , and r . (Triangle Inequality)

其中， $d(p, q)$ 是数据点或数据对象p和q之间的距离或称相异度

- 满足这些属性的距离是度量

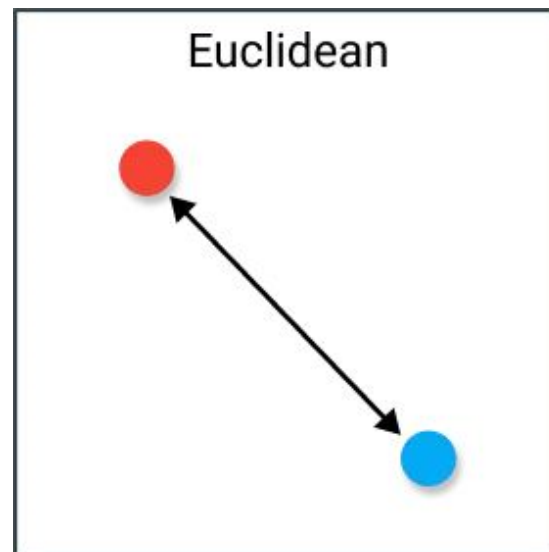
距离度量方法



欧几里得距离

■ 欧几里得距离

$$dist = \sqrt{\sum_{k=1}^n (p_k - q_k)^2}$$



N 是属性个数, p_k 及 q_k 分别是样本 p 及 q 第 k 个属性的取值。

■ 如果范围不同, 则需要标准化.

```
import numpy as np
from scipy.spatial import distance

a = np.array((1, 2, 3))
b = np.array((4, 5, 6))

dist = np.sqrt(np.sum(np.square(a-b)))

print(dist)
print(distance.euclidean(a, b))

5.196152422706632
5.196152422706632
```

余弦相似度

- 如果 d_1 和 d_2 是两个文档向量, 那么

$$\cos(d_1, d_2) = (d_1 \bullet d_2) / \|d_1\| \|d_2\|,$$

其中 $\bullet$ 表示向量的点积, $\|d\|$ 是向量 d 的长度

- 如:

$$d_1 = 3\ 2\ 0\ 5\ 0\ 0\ 0\ 2\ 0\ 0$$

$$d_2 = 1\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 2$$

$$d_1 \bullet d_2 = 3*1 + 2*0 + 0*0 + 5*0 + 0*0 + 0*0 + 0*0 + 2*1 + 0*0 + 0*2 = 5$$

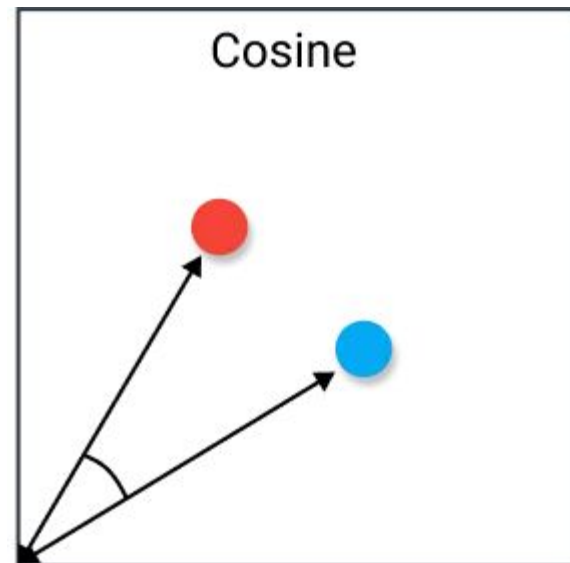
$$\|d_1\| = (3*3 + 2*2 + 0*0 + 5*5 + 0*0 + 0*0 + 0*0 + 2*2 + 0*0 + 0*0)^{0.5} = (42)^{0.5} = 6.481$$

$$\|d_2\| = (1*1 + 0*0 + 0*0 + 0*0 + 0*0 + 0*0 + 0*0 + 1*1 + 0*0 + 2*2)^{0.5} = (6)^{0.5} = 2.245$$

$$\cos(d_1, d_2) = .3150$$

```
from scipy import spatial
List1 = [3, 2, 0, 5, 0, 0, 0, 2, 0, 0]
List2 = [1, 0, 0, 0, 0, 0, 0, 1, 0, 2]
result = 1 - spatial.distance.cosine(List1, List2)
print(result)
```

0.314970394174356



数据质量

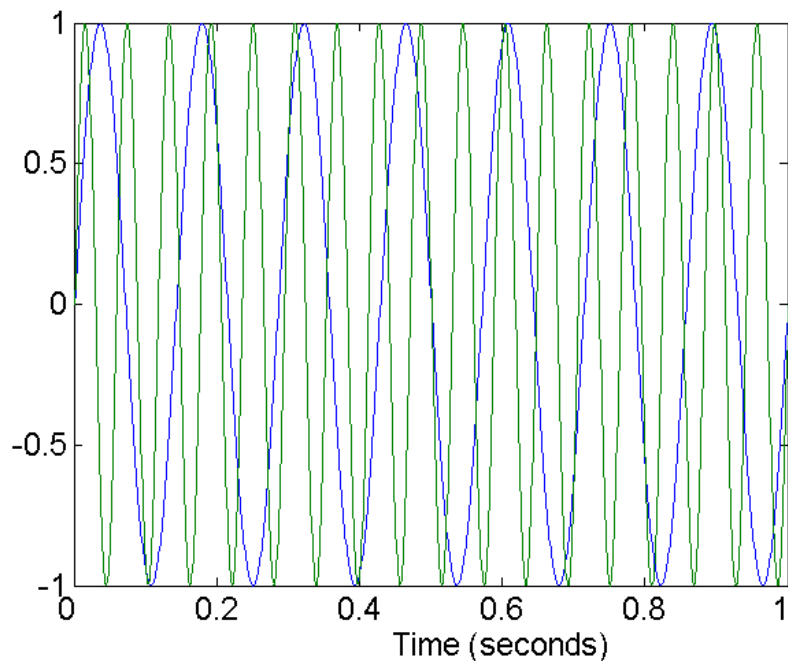
- 什么样的数据质量问题？
- 我们如何检测数据的问题？
- 我们能做些什么来解决这些问题？

- 数据质量问题举例：
 - 噪声和离群点
 - 遗漏值
 - 重复数据

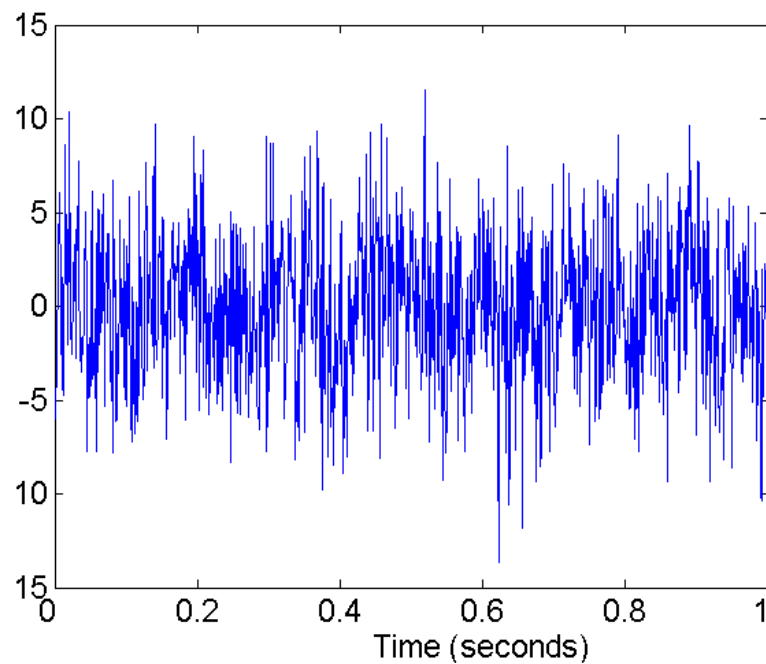
噪声

■ 噪声指原始值的修改

- 如：在破旧的电话上谈话时，人的声音失真，电视屏幕上出现“雪花”



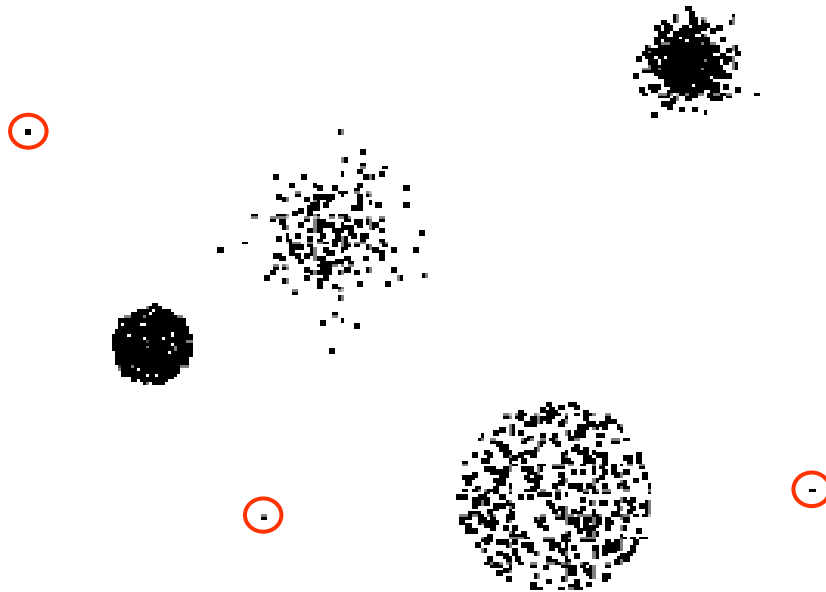
Two Sine Waves



Two Sine Waves + Noise

离群点

- 离群点是具有不同于数据集中其他大部分数据对象的特征的数据对象。



遗漏值

■ 遗漏值的原因

- 没有收集到信息
(如：人们拒绝透露他们的年龄和体重)
- 属性不适用于所有情况
(如：年收入不适用于儿童)

■ 处理遗漏值

- 删除数据对象
- 估计遗漏值
- 在分析期间忽略遗漏值
- 替换为所有可能的值（以其概率加权）

重复数据

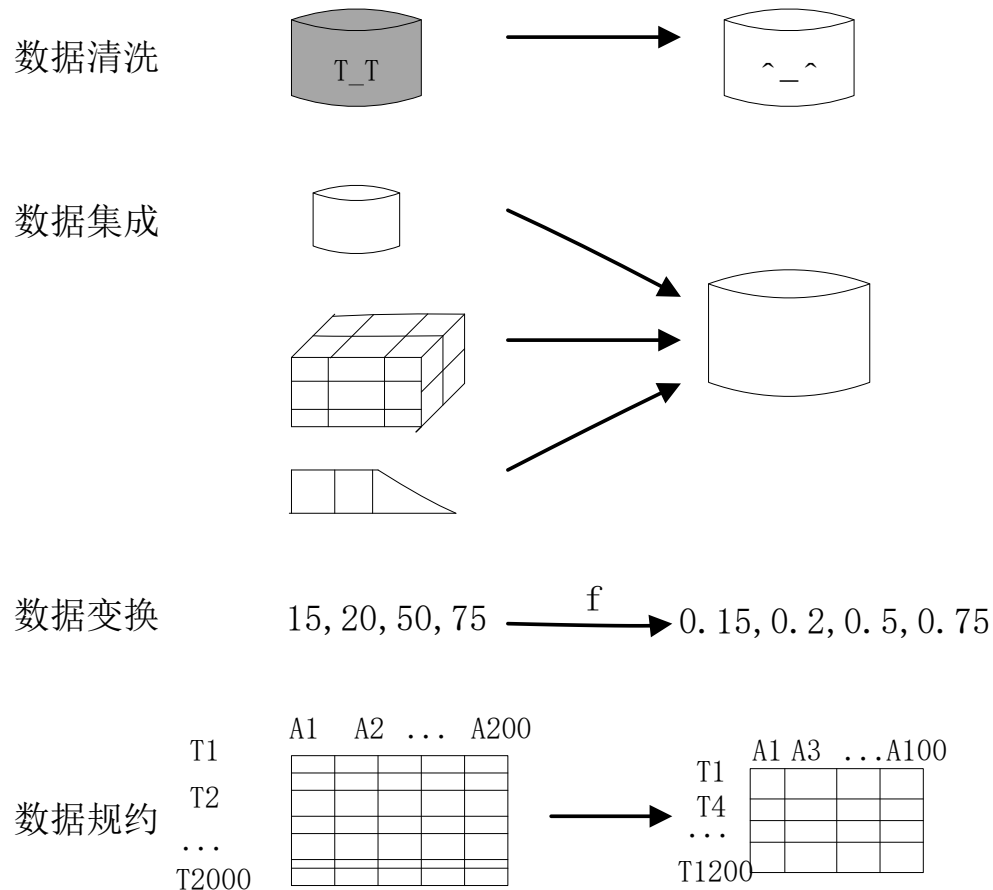
- 数据集可能包含重复或几乎重复的数据对象。
 - 从异构源合并数据时的主要问题

- 如：
 - 一个人有多个电子邮件地址

- 去重复
 - 处理重复数据问题的过程

数据预处理

- 在数据挖掘的过程中，数据预处理占到了整个过程的60%。
- 数据预处理的主要任务包括数据清洗，数据集成，数据变换和数据规约。处理过程如图所示：



- 数据清洗主要是删除原始数据集中的无关数据、重复数据，平滑噪声数据，处理缺失值、异常值等。

缺失值处理

- 处理缺失值的方法可分为三类：删除记录、数据插补和不处理。其中常用的数据插补方法见下表。

插补方法	方法描述
均值/中位数/众数插补	根据属性值的类型，用该属性取值的平均数/中位数/众数进行插补
使用固定值	将缺失的属性值用一个常量替换。如广州一个工厂普通外来务工人员的“基本工资”属性的空缺值可以用 2015 年广州市普通外来务工人员工资标准 1895 元/月，该方法就是使用固定值
最近临插补	在记录中找到与缺失样本最接近的样本的该属性值插补
回归方法	对带有缺失值的变量，根据已有数据和与其有关的其他变量（因变量）的数据建立拟合模型来预测缺失的属性值
插值法	插值法是利用已知点建立合适的插值函数 $f(x)$ ，未知值由对应点 x_i 求出的函数值 $f(x_i)$ 近似代替

缺失值处理

- 插值方法最主要的有拉格朗日插值法和牛顿插值法。以下便对这两种进行介绍。

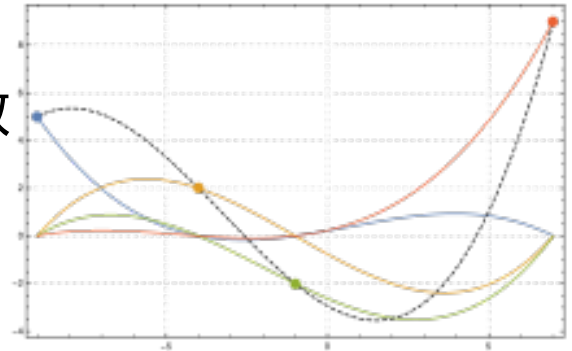
- 拉格朗日插值法

第一步：

求已知的 n 个点 $(x_1, y_1), (x_2, y_2) \cdots (x_n, y_n)$ 的基函数

$$l_i(x) = \prod_{j=1, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

在 x_i 处值为1，在其他的点上取值为0



第二步：

求已知的 n 个点 $(x_1, y_1), (x_2, y_2) \cdots (x_n, y_n)$ 的插值多项式

$$L(x) = \sum_{i=1}^n y_i l_i(x) = \sum_{i=1}^n y_i \prod_{j=1, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

第三步：

将缺失的函数值对应的点 代入插值多项式得到缺失值的近似值 $L(x)$

缺点：当插值点增加或减少一个时，所对应的基本多项式就需要全部重新计算，非常繁琐。

拉格朗日插值法

```
from scipy.interpolate import lagrange # 导入拉格朗日插值函数
from pandas import DataFrame
import numpy as np

# randn() 返回一个20*2的样本, 具有标准正态分布,
df = DataFrame(np.random.randn(20, 2), columns=['first', 'second'])

print(df)
print("-----df-----")

df[(df['first'] < -1) | (df['first'] > 1)] = None # 将异常值变成空值

print(df)
print("-----df-----")

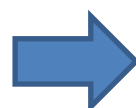
def ployinterp_column(s, n, k=5):
    # 取数, n的前后5个, 这里有可能取到不存在的下标, 为空
    start = 0 if n - k < 0 else n - k
    if n == len(s) - 1:
        y = s[list(range(start, n))]
    else:
        end = n + 1 + k
        end = end if end < len(s) else len(s) - 1
        y = s[list(range(start, n)) + list(range(n + 1, end))]

    y = y[y.notnull()] # 剔除空值
    return lagrange(y.index, list(y))(n) # 插值并返回插值结果, 最后的括号就是要插值的n

# 逐个判断是否需要插值
for i in df.columns:
    for j in range(len(df)): # 逐个元素判断
        if (df[i].isnull())[j]: # 如果为空即插值
            df[i][j] = ployinterp_column(df[i], j)
print(df)
```

缺失值处理

	first	second
0	-0.225771	0.158400
1	-0.974101	1.010136
2	-0.564684	-0.114466
3	0.900858	1.203132
4	0.350937	-0.814911
5	NaN	NaN
6	NaN	NaN
7	0.958246	1.311487
8	0.549183	1.171870
9	0.632100	1.642590
10	-0.259342	1.524784
11	-1.083484	1.048618
12	-0.195943	-0.046190
13	-0.899700	0.702885
14	-0.459151	1.637190
15	0.175222	0.075272
16	-0.515593	-0.343457
17	-0.257240	-1.176515
18	-0.516034	-1.538261
19	-0.808186	-1.662986



	first	second
0	-0.225771	0.158400
1	-0.974101	1.010136
2	-0.564684	-0.114466
3	0.900858	1.203132
4	0.350937	-0.814911
5	-0.577955	-2.978023
6	0.186557	-1.098391
7	0.958246	1.311487
8	0.549183	1.171870
9	0.632100	1.642590
10	-0.259342	1.524784
11	-1.083484	1.048618
12	-0.195943	-0.046190
13	-0.899700	0.702885
14	-0.459151	1.637190
15	0.175222	0.075272
16	-0.515593	-0.343457
17	-0.257240	-1.176515
18	-0.516034	-1.538261
19	-0.808186	-1.662986

异常值处理

- 在数据预处理时，异常值是否剔除，需视具体情况而定，因为有些异常值可能蕴含着有用的信息。异常值处理常用方法见下表：

异常值处理方法	方法描述
删除含有异常值的记录	直接将含有异常值的记录删除。
视为缺失值	将异常值视为缺失值，利用缺失值处理的方法进行处理。
平均值修正	可用前后两个观测值的平均值修正该异常值。
不处理	直接在具有异常值的数据集上进行挖掘建模。

噪声数据处理

■分箱 (Binning) ——噪声光滑

- 首先排序数据，并将他们分到等深的箱中（每箱记录数称为箱子深度）
- 然后可以按箱的平均值平滑、按箱中值平滑、按箱的边界平滑等等

■聚类 ——去除孤立点

- 监测并且去除孤立点

■回归 ——平滑数据

- 通过让数据适应回归函数来平滑数据

数据平滑的分箱方法

□ price的排序后数据（单位：美元）：4, 8, 15, 21, 21, 24, 25, 28, 34

□ 划分为（等深的）箱：

■ 箱1：4, 8, 15

■ 箱2：21, 21, 24

■ 箱3：25, 28, 34

□ 用箱平均值平滑(或者: 中位数)

■ 箱1：9, 9, 9

■ 箱2：22, 22, 22

■ 箱3：29, 29, 29

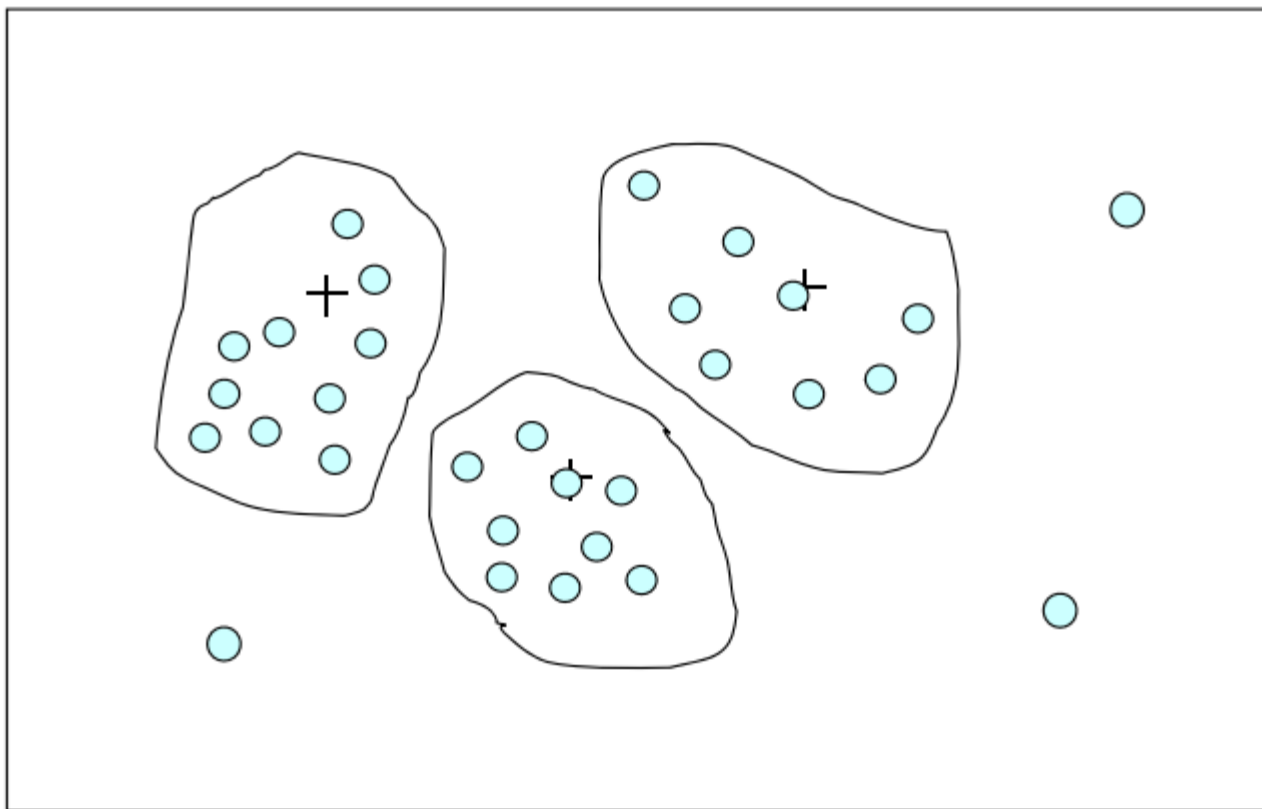
□ 用箱边界平滑：

■ 箱1：4, 4, 15

■ 箱2：21, 21, 24

■ 箱3：25, 25, 34

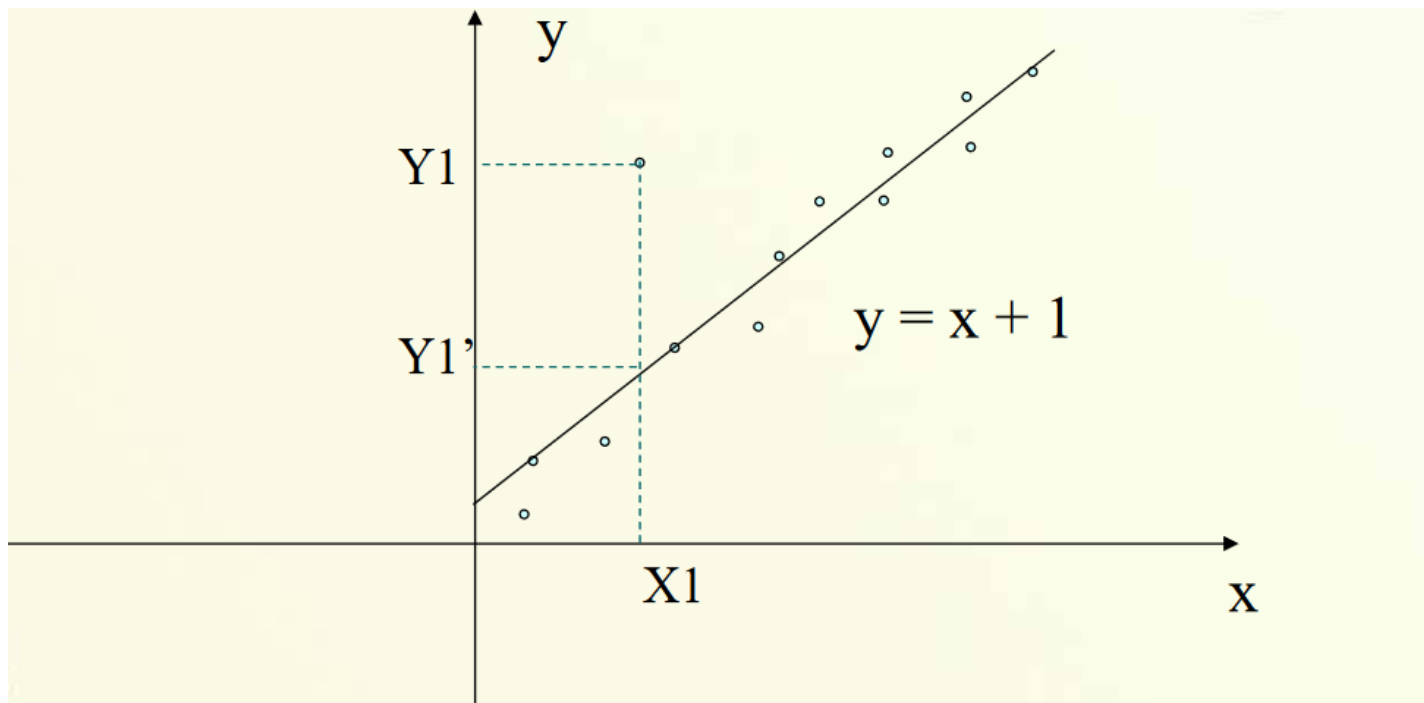
聚类分析去除噪声数据



□ 通过聚类分析查找孤立点，消除噪声

回归分析去除噪声数据

回归: 用一个(回归)函数拟合数据来光滑



- 数据挖掘需要的数据往往分布在不同的数据源中，数据集成就是将多个数据源合并存放在一个一致的数据存储（如数据仓库）中的过程。
- 在数据集成时，来自多个数据源的现实世界实体的表达形式是不一样的，不一定是匹配的，要考虑实体识别问题和属性冗余问题，从而把源数据在最低层上加以转换、提炼和集成。

- 实体识别的任务是检测 and 解决同名异义、异名同义、单位不统一的冲突。
如：
 - **同名异义**：数据源A中的属性ID和数据源B中的属性ID分别描述的是菜品编号和订单编号，即描述的是不同的实体。
 - **异名同义**：数据源A中的sales_dt和数据源B中的sales_date都是描述销售日期的，即A. sales_dt = B. sales_date。
 - **单位不统一**：描述同一个实体分别用的是国际单位和中国传统的计量单位。

数据集成

将数据表中的某一列作为键值，当键值相同时放在同一行，相当于将该行键值合并

Pands merge方法, merge(left, right, how="inner"...)

连接方式how, inner、left、right、outer，默认为inner

Inner: 根据键值匹配；left，保留左表全部配数据；right，保留右表全部配数据；outer，保留全部未匹配数据

```
import pandas as pd
import numpy as np

df1=pd.DataFrame({'姓名':['小红','小风','小黑'],\
                  '年龄':['18','15','17'],\
                  '性别':['女','男','男']})
df2=pd.DataFrame({'姓名':['大红','小风','大黑','小黑'],\
                  'age':['28','25','27','25'],\
                  'gender':['女','男','男','女']})
print('\n按照==左边==列表显示-----')
merg=pd.merge(left=df1,right=df2,how="left",\
              left_on="姓名",right_on="姓名",)
print(merg)
print('\n按照==右边===列表显示-----')
merg1=pd.merge(left=df1,right=df2,how="right",\
              left_on="姓名",right_on="姓名",)
print(merg1)
print('\n按照===交===显示-----')
merg2=pd.merge(left=df1,right=df2,how="inner",\
              left_on="姓名",right_on="姓名",)
print(merg2)
print('\n按照===并===显示-----')|
merg3=pd.merge(left=df1,right=df2,how="outer",\
              left_on="姓名",right_on="姓名",)
print(merg3)
```

按照==左边==列表显示-----

	姓名	年龄	性别	age	gender
0	小红	18	女	NaN	NaN
1	小风	15	男	25	男
2	小黑	17	男	25	女

按照==右边===列表显示-----

	姓名	年龄	性别	age	gender
0	大红	NaN	NaN	28	女
1	小风	15	男	25	男
2	大黑	NaN	NaN	27	男
3	小黑	17	男	25	女

按照===交===显示-----

	姓名	年龄	性别	age	gender
0	小风	15	男	25	男
1	小黑	17	男	25	女

按照===并===显示-----

	姓名	年龄	性别	age	gender
0	小红	18	女	NaN	NaN
1	小风	15	男	25	男
2	小黑	17	男	25	女
3	大红	NaN	NaN	28	女
4	大黑	NaN	NaN	27	男

数据集成——冗余属性识别

- 数据集成往往导致数据冗余，如：
 - 同一属性多次出现
 - 同一属性命名不一致导致重复
- 不同源数据的仔细整合能减少甚至避免数据冗余与不一致，以提高数据挖掘的速度和质量。对于冗余属性要先分析检测到后再将其删除。
- 有些冗余属性可以用相关分析检测到。给定两个数值型的属性A和B，根据其属性值，可以用相关系数度量一个属性在多大程度上蕴含另一个属性。

- 主要是对数据进行规范化的操作，将数据转换成“适当的”格式，以适用于挖掘任务及算法的需要。

数据变换——简单函数变换

- 简单函数变换就是对原始数据进行某些数学函数变换，常用的函数变换包括平方、开方、对数、差分运算等，即：

$$x' = x^2$$

$$x' = \sqrt{x}$$

$$x' = \log(x)$$

$$\nabla f(x_k) = f(x_{k+1}) - f(x_k)$$

```
import math
print("平方结果",math.pow(12, 2))
print("开方结果",math.sqrt(144))
print("对数结果",math.log(144))
```

平方结果 144.0
开方结果 12.0
对数结果 4.969813299576001

```
import numpy as np
import pandas as pd

b = pd.DataFrame(list(range(1, 5)))

b_diff_1 = b.diff(periods=1)
b_diff_2 = b.diff(periods=2)

print(b)
print(b_diff_1)
print(b_diff_2)
```

```
      0
0  1
1  2
2  3
3  4
      0
0  NaN
1  1.0
2  1.0
3  1.0
      0
0  NaN
1  NaN
2  2.0
3  2.0
```

- 数据标准化（归一化）处理是数据挖掘的一项基础工作，不同评价指标往往具有不同的量纲和量纲单位，数值间的差别可能很大，不进行处理可能会影响到数据分析的结果，为了消除指标之间的量纲和大小不一的影响，需要进行数据标准化处理，将数据按照比例进行缩放，使之落入一个特定的区域，从而进行综合分析。如将工资收入属性值映射到 $[-1, 1]$ 或者 $[0, 1]$ 之间。
- 下面介绍三种规范化方法：最小-最大规范化、零-均值规范化、小数定标规范化

数据变换——规范化

- 最小-最大规范化：也称为离差标准化，是对原始数据的线性变换，使结果值映射到[0,1]之间。

转换函数如：

$$x^* = \frac{x - \min}{\max - \min}$$

其中 $\max$ 为样本数据的最大值, $\min$ 为样本数据的最小值。

```
from sklearn import preprocessing
import numpy as np
#初始化数据，每一行表示一个样本，每一列表示一个特征
x=np.array([[0.,-3.,1.],[3.,1.,2.],[0.,1.,-1.]])
#最小-最大规范化
min_max_scaler = preprocessing.MinMaxScaler()#获得一个转换器对象
minmax_x=min_max_scaler.fit_transform(x)#使用转换器的方法进行转换（按列标准化）
print(minmax_x)
```

```
array([[0.          , 0.          , 0.66666667],
       [1.          , 1.          , 1.          ],
       [0.          , 1.          , 0.          ]])
```

数据变换——规范化

零-均值规范化:也叫标准差标准化, 经过处理的数据的平均数为0, 标准差为1。转化函数为:

$$x^* = \frac{x - \bar{x}}{\sigma}$$

其中 $\bar{x}$ 为原始数据的均值, σ 为原始数据的标准差。

#零-均值规范化

```
scaled_x = preprocessing.scale(x)  
print(scaled_x)
```

```
array([[ -0.70710678,  -1.41421356,   0.26726124],  
       [  1.41421356,   0.70710678,   1.06904497],  
       [ -0.70710678,   0.70710678,  -1.33630621]])
```

数据变换——规范化

小数定标规范化:通过移动属性值的小数位，将属性值映射到 $[-1, 1]$ 之间，移动的小数位取决于属性值绝对值的最大值。转化函数为：

$$x^* = \frac{x}{10^k}$$

```
1  # coding:utf-8
2  import numpy as np
3  # 初始化数据
4  x = np.array([[ 0., -3.,  1.],
5                [ 3.,  1.,  2.],
6                [ 0.,  1., -1.]])
7  # 小数定标规范化
8  j = np.ceil(np.log10(np.max(abs(x)))) # 获取小数点移动位数
9  scaled_x = x/(10**j)
10 print scaled_x
```

```
array([[ 0. , -0.3,  0.1],
       [ 0.3,  0.1,  0.2],
       [ 0. ,  0.1, -0.1]])
```

数据变换——连续属性离散化

- 一些数据挖掘算法，特别是某些分类算法，要求数据是分类属性形式，如ID3算法等。这样，常常需要将连续属性变换成分类属性，即连续属性离散化。

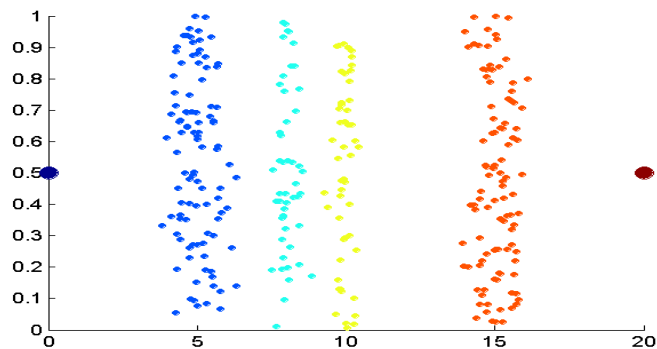
➤ 离散化的过程

连续属性变换成分类属性涉及两个子任务：决定需要多少个分类变量，以及确定如何将连续属性值映射到这些分类值。

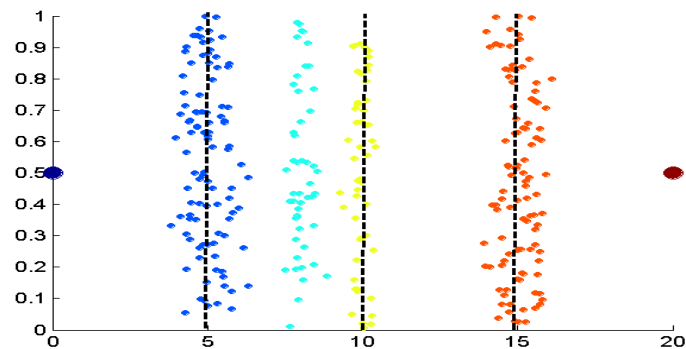
➤ 常用的离散化方法

常用的无监督（不使用标签）离散化方法有：等宽法、等频法、基于聚类分析的方法

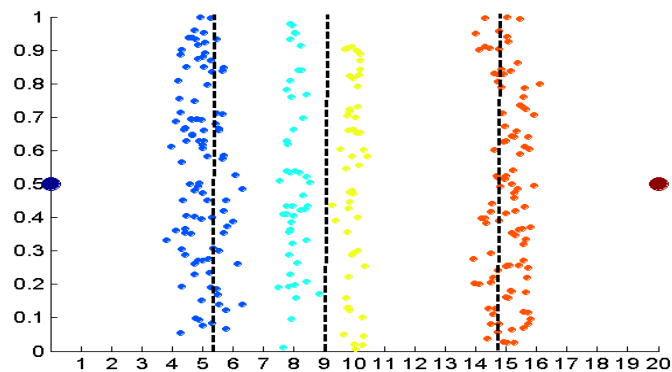
不使用类标签进行离散化



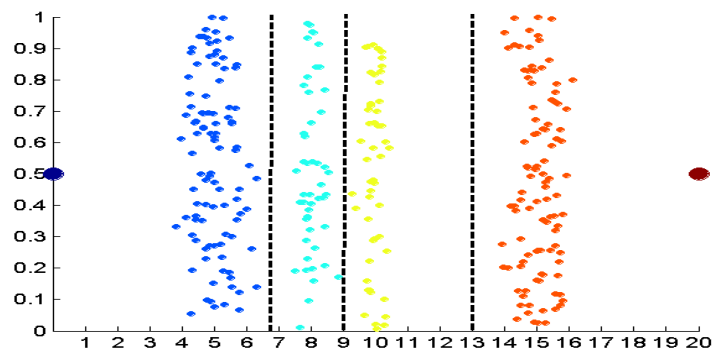
Data



Equal interval width



Equal frequency



K-means

- 数据规约是将海量数据进行规约，规约之后的数据仍接近于保持原数据的完整性，但数据量小得多。
- 通过数据规约，可以达到：
 - 降低无效、错误数据对建模的影响，提高建模的准确性
 - 少量且具代表性的数据将大幅缩减数据挖掘所需的时间
 - 降低储存数据的成本

数据规约——属性规约

属性规约方法	方法描述
合并属性	将一些旧属性合并为新属性
逐步向前选择	从一个空属性集开始，每次从原来属性集合中选择一个当前最优的属性添加到当前属性子集中，直到无法选出最优属性或满足一定阈值约束为止
逐步向后删除	从全属性集开始，每次从当前属性子集中选择一个当前最差属性并将其从当前属性子集中消去，与上面相反
决策树归纳	利用决策树的归纳方法对初始数据进行分类归纳学习，获得一个初始决策树，所有没有出现在决策树上的属性均可认为是无关属性，因此可以将这些属性删除
主成分分析	用较少的变量去解释原数据中的大部分变量，将许多相关性很高的变量转化成彼此相互独立或不相关变量

数据规约——属性规约

下面详细介绍**主成分分析**计算步骤：

1) 设原始变量 $X_1, X_2, \dots, X_p$ 的观测 n 次数据矩阵为：

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix} \\ \triangleq (X_1, X_2, \dots, X_p)$$

2) 将数据矩阵**中心化**。为了方便，将标准化后的数据矩阵仍然记为 X 。

3) 求相关系数矩阵 $R = (r_{ij})_{p \times p}$ 的定义为：

$$r_{ij} = \sum_{k=1}^n (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j) / \sqrt{\sum_{k=1}^n (x_{ki} - \bar{x}_i)^2 \sum_{k=1}^n (x_{kj} - \bar{x}_j)^2} \quad \text{其中 } r_{ij} = r_{ji}, r_{ii} = 1$$

4) 求 R 的特征方程 $\det(R - \lambda E) = 0$ 的特征根 $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p > 0$ 。

5) 确定主成分个数 m : $\sum_{i=1}^m \lambda_i / \sum_{i=1}^p \lambda_i \geq \alpha$ ， α 根据实际问题确定，一般取80%。

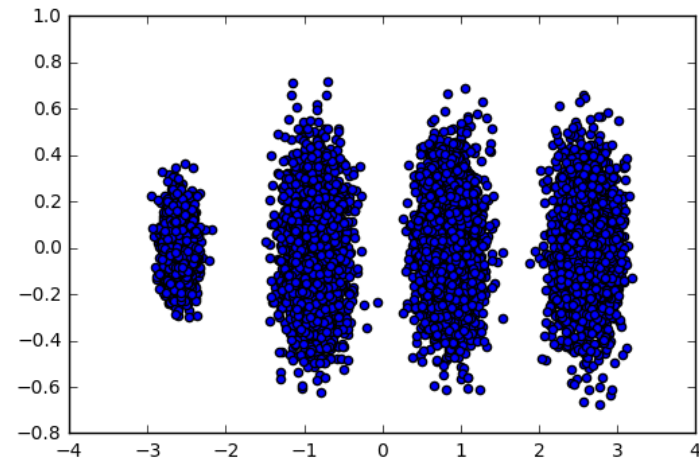
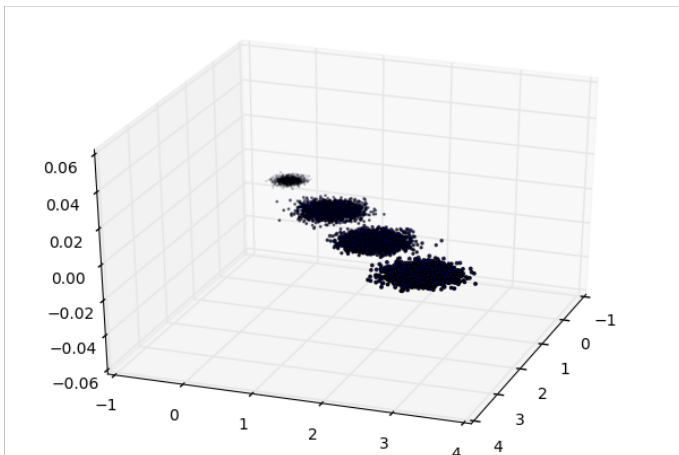
6) 计算 m 个相应的单位特征向量:

$$\beta_1 = \begin{bmatrix} \beta_{11} \\ \beta_{21} \\ \vdots \\ \beta_{p1} \end{bmatrix}, \beta_2 = \begin{bmatrix} \beta_{12} \\ \beta_{22} \\ \vdots \\ \beta_{p2} \end{bmatrix}, \dots, \beta_m = \begin{bmatrix} \beta_{1m} \\ \beta_{2m} \\ \vdots \\ \beta_{pm} \end{bmatrix}$$

7) 计算主成分:

$$Z_i = \beta_{1i}X_1 + \beta_{2i}X_2 + \dots + \beta_{pi}X_p \quad i = 1, 2, \dots, m$$

`sklearn.decomposition.PCA`



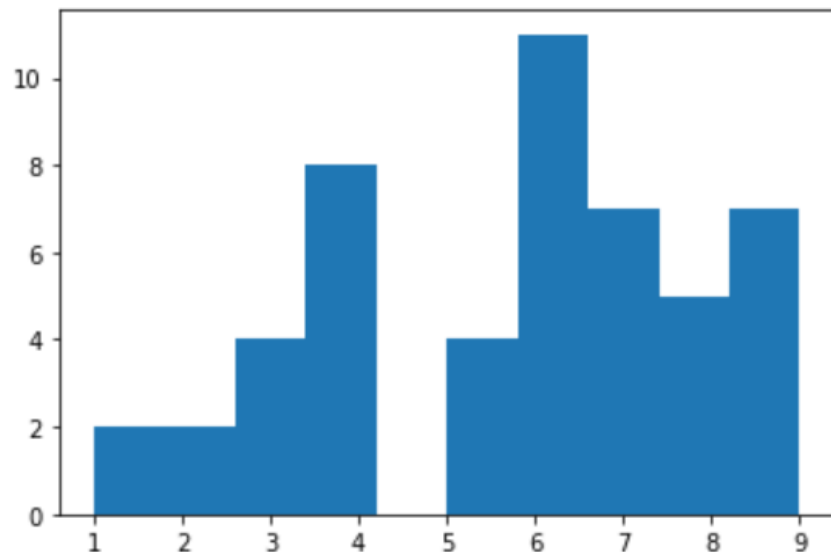
- 数值规约通过选择替代的、较小的数据来减少数据量。数值规约可以是有参的，也可以是无参的。有参方法是使用一个模型来评估数据，只需存放参数，而不需要存放实际数据。有参的数值规约技术主要有两种：回归（线性回归和多元回归）和对数线性模型（近似离散属性集中的多维概率分布）。数值规约常用方法有：
 - 直方图
 - 用聚类数据表示实际数据
 - 抽样（采样）
 - 参数回归法。

数据规约——数值规约

以直方图为例

表现：用分箱表示数据分布，每个箱子代表一个属性-频率对

```
#直方图  
#以50个随机整数为例  
import numpy as np  
data = np.random.randint(1, 10, 50)  
import matplotlib.pyplot as plt  
plt.hist(data)  
plt.show()
```



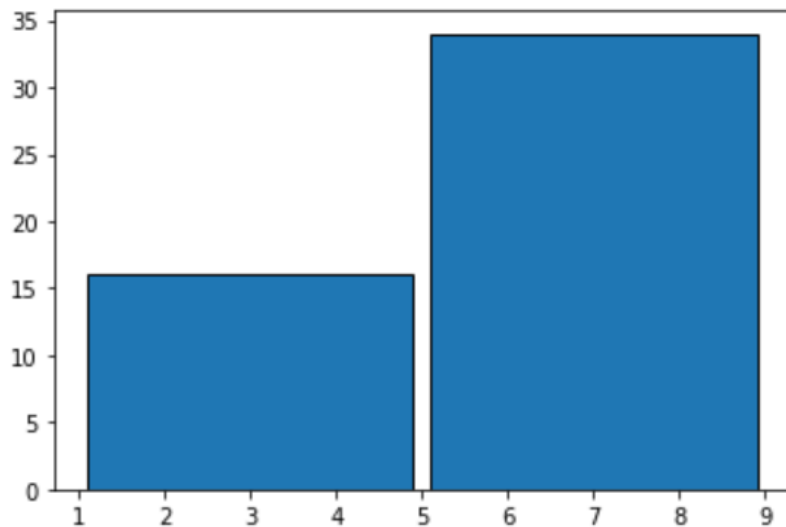
直方图每一个箱子代表了属性-频率对，例如属性2对应的箱子高度（Y值）为2，表示2在数据集中出现了2次。

数据规约——数值规约

以直方图为例

表现：用分箱表示数据分布，每个箱子代表一个属性-频率对

```
bins = np.linspace(data.min(), data.max(), 3, endpoint = True)
##out: bins = array([1., 5., 9.]), 基于这三个点把区间分为[1, 5), [5, 9]
#利用bins结果重新生成直方图
plt.hist(data, bins = bins, rwidth = 0.95, edgecolor = 'k')
plt.show()
```



Thank You!